

Subject 5: Production-Grade RAG System Design & Failure-Mode Diagnosis

1. The 80/20 Core (Hiring-Critical)

What Actually Matters (20% → 80% Impact)

Included (Production & Interview Critical):

1. **Retrieval Failure Diagnosis** (35% of value)
 - Systematic debugging framework for "wrong results"
 - Query-document mismatch analysis
 - Embedding quality assessment
 - Index health monitoring
 - Why: 80% of RAG failures are retrieval problems, not generation problems
2. **Reranking & Filtering Strategies** (25% of value)
 - Cross-encoder reranking for precision
 - MMR (Maximum Marginal Relevance) for diversity
 - Metadata filtering for precision
 - Hybrid search (vector + keyword)
 - Why: Initial retrieval is noisy; reranking doubles precision
3. **Context Window Optimization** (20% of value)
 - Chunk size vs. retrieval precision tradeoffs
 - Dynamic top-k selection
 - Context compression techniques
 - Relevance-based truncation
 - Why: Token costs and quality depend on optimal context selection
4. **Query Transformation Techniques** (10% of value)
 - Query expansion and rewriting
 - Hypothetical document embeddings (HyDE)
 - Multi-query strategies
 - Why: User queries are often poor search queries
5. **RAG-Specific Evaluation Metrics** (10% of value)
 - Retrieval metrics (MRR, NDCG, Recall@k)
 - Generation metrics (faithfulness, relevance)

- End-to-end metrics (answer quality)
- Why: "It works" isn't measurable; metrics enable optimization

Explicitly Excluded (Low ROI / Interview Anti-Signals):

- ❌ Fine-tuning embedding models (unless at scale)
 - *Why excluded:* Overkill for most use cases; start with off-the-shelf
- ❌ Complex graph-based retrieval (knowledge graphs)
 - *Why excluded:* Premature optimization; vector search + reranking solves 90%
- ❌ Custom vector databases from scratch
 - *Why excluded:* Use Chroma, Pinecone, Qdrant; don't reinvent
- ❌ Retrieval-only optimization (ignoring generation)
 - *Why excluded:* End-to-end quality matters, not just retrieval precision
- ❌ "Just increase top-k" as solution to quality issues
 - *Why excluded:* Wastes tokens, adds noise; proper fix is better retrieval

Hiring Signal Justification

What Tier 1 interviewers look for:

- Can you debug why RAG returns wrong answers systematically?
- Do you understand retrieval precision vs. recall tradeoffs?
- Have you implemented reranking or hybrid search?
- Can you measure RAG quality with concrete metrics?
- Do you know when RAG fails and how to fix it?

What signals "student not engineer":

- "Just use LangChain defaults" (no understanding of internals)
 - No retrieval metrics (can't measure quality)
 - Treating vector search as black box
 - Not understanding chunking impact on retrieval
 - No failure mode taxonomy (every failure is "bad results")
-

2. Genius-Level Understanding

Core Mental Models

Perspective 1: Infrastructure Engineer

RAG is a cache miss problem. Optimize for cache hits (retrieval quality).

Traditional caching:

python

```
def get_user(user_id):
    # Check cache
    cached = redis.get(f"user:{user_id}")
    if cached:
        return cached # Cache hit

    # Cache miss - fetch from DB
    user = db.query(f"SELECT * FROM users WHERE id={user_id}")
    redis.set(f"user:{user_id}", user)
    return user
```

RAG as semantic cache:

python

```
def answer_question(query):
    # Semantic cache lookup (retrieval)
    relevant_docs = vector_db.search(query, top_k=5)

    if relevant_docs: # "Cache hit"
        # Generate from cached knowledge
        answer = llm.complete(
            f"Context: {relevant_docs}\nQuestion: {query}\nAnswer:"
        )
        return answer
    else: # "Cache miss"
        # No relevant knowledge found
        return "I don't have information to answer this."
```

Key insight: Cache hit rate = retrieval quality. Optimize retrieval like you'd optimize cache hits.

Production implications:

python

```
# Amateur: No retrieval monitoring
def rag_query(query):
    docs = vector_db.search(query, top_k=5)
    return llm.complete(f"{docs}\n{query}")

# Engineer: Monitor retrieval quality
class MonitoredRAG:
    def __init__(self):
        self.metrics = {
            'queries_with_results': 0,
            'queries_without_results': 0,
            'avg_top1_score': [],
            'avg_top5_score': []
        }

    def query(self, query):
        # Retrieve with scores
        results = vector_db.search(query, top_k=5, return_scores=True)

        # Monitor retrieval quality
        if not results:
            self.metrics['queries_without_results'] += 1
            logger.warning(f"No results for: {query}")
            return "I don't have relevant information."

        self.metrics['queries_with_results'] += 1
        self.metrics['avg_top1_score'].append(results[0].score)
        self.metrics['avg_top5_score'].append(
            sum(r.score for r in results) / len(results)
        )

        # Alert if retrieval quality degrading
        if results[0].score < 0.5:
            logger.warning(
                f"Low relevance score ({results[0].score:.3f}) for: {query}"
            )

        # Generate answer
        context = "\n\n".join([r.text for r in results])
        return llm.complete(f"Context: {context}\nQ: {query}\nA:")
```

```
def get_retrieval_health(self):
    return {
        'cache_hit_rate': self.metrics['queries_with_results'] /
            (self.metrics['queries_with_results'] +
             self.metrics['queries_without_results']),
        'avg_relevance': np.mean(self.metrics['avg_top1_score']),
        'total_queries': len(self.metrics['avg_top1_score'])
    }
```

Why this matters: Without monitoring, you can't tell if RAG is working or failing silently.

Perspective 2: Product Engineer

RAG failure modes map to user experiences. Understand the user-facing symptoms.

Failure Mode Taxonomy:

python

```
class RAGFailureMode(Enum):
    # Retrieval failures
    NO_RELEVANT_DOCS = "retrieval_empty"           # No results found
    LOW_RELEVANCE = "retrieval_weak"               # Results not relevant
    MISSING_KEY_INFO = "retrieval_incomplete"      # Missing critical chunks
    TOO_MANY_RESULTS = "retrieval_noisy"           # Top-k too high, noise

    # Generation failures
    HALLUCINATION = "generation_hallucination"      # LLM ignores context
    CONTEXT_OVERFLOW = "generation_truncated"       # Context too long
    ANSWER_NOT_GROUNDED = "generation_unfaithful"  # Answer not from docs

    # System failures
    LATENCY_TIMEOUT = "system_timeout"             # Retrieval too slow
    COST_OVERRUN = "system_expensive"              # Too many tokens

class RAGDiagnostics:
    def diagnose_failure(self, query, retrieved_docs, answer, expected_answer=None):
        """Diagnose what went wrong"""

        # Check retrieval
        if not retrieved_docs:
            return RAGFailureMode.NO_RELEVANT_DOCS

        top_score = retrieved_docs[0].score
        if top_score < 0.5:
```

```

        return RAGFailureMode.LOW_RELEVANCE

    # Check if answer is grounded in context
    if not self.is_answer_grounded(answer, retrieved_docs):
        return RAGFailureMode.ANSWER_NOT_GROUNDED

    # Check if key information is missing
    if expected_answer and not self.covers_expected_info(retrieved_docs, expected_answer):
        return RAGFailureMode.MISSING_KEY_INFO

    return None # No obvious failure

def is_answer_grounded(self, answer, docs):
    """Check if answer uses info from docs"""
    # Extract key entities/facts from answer
    answer_facts = self.extract_facts(answer)

    # Check if facts appear in docs
    doc_text = " ".join([d.text for d in docs])
    grounded_facts = [
        fact for fact in answer_facts
        if self.fact_appears_in_text(fact, doc_text)
    ]

    grounding_rate = len(grounded_facts) / len(answer_facts)
    return grounding_rate > 0.8 # 80% of facts must be grounded

```

User-facing symptoms:

python

```

SYMPTOMS = {
    RAGFailureMode.NO_RELEVANT_DOCS:
        "Bot says 'I don't have information' when docs clearly exist",

    RAGFailureMode.LOW_RELEVANCE:
        "Bot gives generic answers unrelated to the specific question",

    RAGFailureMode.MISSING_KEY_INFO:
        "Bot's answer is partially correct but missing important details",

    RAGFailureMode.HALLUCINATION:
        "Bot confidently states facts not in the knowledge base",

    RAGFailureMode.CONTEXT_OVERFLOW:

```

```

    "Bot's answer is cut off mid-sentence",

    RAGFailureMode.ANSWER_NOT_GROUNDED:
        "Bot's answer contradicts the source documents"
}

```

How this helps:

- Users report symptoms ("answer is incomplete")
- Engineer maps to failure mode (MISSING_KEY_INFO)
- Applies appropriate fix (improve chunking, increase top-k)

Perspective 3: Researcher

RAG is a two-stage pipeline: retrieval (information recall) → generation (information synthesis). Optimize each independently.

Mathematical formulation:

```

RAG Quality = f(Retrieval Precision, Retrieval Recall, Generation Faithfulness)

```

Where:

- Precision = % of retrieved docs that are relevant
- Recall = % of relevant docs that were retrieved
- Faithfulness = % of answer that comes from retrieved docs

The Precision-Recall Tradeoff:

python

```

# Low top-k (k=3)
precision = 0.9 # 90% of retrieved docs are relevant
recall = 0.4    # But only got 40% of relevant docs
# Result: Accurate but incomplete answers

# High top-k (k=20)
precision = 0.3 # 30% of retrieved docs are relevant
recall = 0.8    # Got 80% of relevant docs
# Result: Complete but noisy answers (LLM distracted)

# Optimal (reranking)
# Retrieve k=20, rerank to top-5
initial_recall = 0.8

```

```
reranked_precision = 0.85
# Result: Complete AND accurate
```

Advanced insight: Query-document embedding space mismatch

python

```
# Problem: Questions and answers live in different semantic spaces

query = "How do I reset my password?"
# Embedding: [0.2, 0.5, 0.8, ...]

doc = "To reset your password, click the 'Forgot Password' link..."
# Embedding: [0.1, 0.4, 0.3, ...]

# Cosine similarity might be low even though doc answers query!
# Why? Query is a question, doc is an instruction.
```

Solution 1: Query transformation (HyDE)

python

```
# Generate hypothetical answer
hypothetical_answer = llm.complete(
    f"Answer this question: {query}"
)
# "You can reset your password by clicking..."

# Embed the hypothetical answer (more similar to docs)
query_embedding = embed(hypothetical_answer)

# Now similarity is higher
results = vector_db.search(query_embedding, top_k=5)
```

Solution 2: Asymmetric embeddings

python

```
# Use different embedding prefixes
query_embedding = embed(f"query: {query}")
doc_embedding = embed(f"passage: {doc}")

# Or use asymmetric models (e.g., E5, BGE)
```


Perspective 4: Hiring Manager

Show me you've debugged production RAG failures and measured improvements.

Red flags:

- "RAG works in my tests" (no production failure experience)
- Can't explain why retrieval failed
- No retrieval metrics (MRR, NDCG, Recall@k)
- No reranking or filtering
- Treating RAG as black box

Green flags:

- Systematic failure diagnosis (taxonomy of failure modes)
 - A/B tested retrieval strategies
 - Reranking implementation with measured improvement
 - Retrieval monitoring dashboard
 - Cost/latency optimization data
-

Advanced Production Use Cases

Case 1: Hybrid Search (Vector + Keyword)

Problem: Vector search misses exact matches (product IDs, names, codes).

Solution: Combine vector similarity with keyword (BM25) search.

python

```
from rank_bm25 import BM25Okapi
import numpy as np

class HybridRetriever:
    def __init__(self, vector_db, documents):
        self.vector_db = vector_db
        self.documents = documents

        # Build BM25 index
        tokenized_docs = [doc.split() for doc in documents]
        self.bm25 = BM25Okapi(tokenized_docs)
```

```

def search(self, query, top_k=10, alpha=0.5):
    """
    Hybrid search combining vector and keyword

    alpha: weight for vector search (0-1)
    1-alpha: weight for keyword search
    """
    # Vector search
    vector_results = self.vector_db.search(query, top_k=top_k*2)
    vector_scores = {r.id: r.score for r in vector_results}

    # Keyword search (BM25)
    tokenized_query = query.split()
    bm25_scores = self.bm25.get_scores(tokenized_query)

    # Normalize scores to [0, 1]
    def normalize(scores):
        min_s, max_s = min(scores), max(scores)
        if max_s == min_s:
            return scores
        return [(s - min_s) / (max_s - min_s) for s in scores]

    vector_scores_norm = normalize(list(vector_scores.values()))
    bm25_scores_norm = normalize(bm25_scores)

    # Combine scores
    combined_scores = {}
    for i, doc_id in enumerate(range(len(self.documents))):
        v_score = vector_scores_norm[i] if doc_id in vector_scores else 0
        k_score = bm25_scores_norm[i]

        combined_scores[doc_id] = alpha * v_score + (1 - alpha) * k_score

    # Return top-k
    ranked = sorted(
        combined_scores.items(),
        key=lambda x: x[1],
        reverse=True
    )[:top_k]

    return [self.documents[doc_id] for doc_id, score in ranked]

# Usage
retriever = HybridRetriever(vector_db, documents)

# Query with exact product code

```

```

results = retriever.search("SKU-12345 installation guide", alpha=0.3)
# Low alpha = more weight on keywords (catches exact match)

# Conceptual query
results = retriever.search("how to improve performance", alpha=0.7)
# High alpha = more weight on vectors (semantic understanding)

```

Impact: 15-25% improvement in recall for queries with exact matches.

When to use:

- Product catalogs (SKUs, model numbers)
- Technical documentation (error codes, API names)
- Legal documents (case numbers, citations)

Case 2: Cross-Encoder Reranking

Problem: Vector search is fast but imprecise. It ranks by embedding similarity, not true relevance.

Solution: Use cross-encoder to rerank top-k results.

python

```

from sentence_transformers import CrossEncoder

class RerankedRetriever:
    def __init__(self, vector_db):
        self.vector_db = vector_db

        # Load cross-encoder model (slow but accurate)
        self.reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

    def search(self, query, initial_k=20, final_k=5):
        """
        Two-stage retrieval:
        1. Fast vector search (get initial_k candidates)
        2. Slow reranking (rerank to final_k results)
        """
        # Stage 1: Vector search (fast, recall-optimized)
        candidates = self.vector_db.search(query, top_k=initial_k)

        if not candidates:
            return []

```

```

# Stage 2: Cross-encoder reranking (slow, precision-optimized)
pairs = [[query, c.text] for c in candidates]
rerank_scores = self.reranker.predict(pairs)

# Combine candidates with rerank scores
reranked = [
    (candidate, score)
    for candidate, score in zip(candidates, rerank_scores)
]

# Sort by rerank score
reranked.sort(key=lambda x: x[1], reverse=True)

# Return top final_k
return [candidate for candidate, score in reranked[:final_k]]

# Comparison
vector_only = vector_db.search("Python memory management", top_k=5)
reranked = reranked_retriever.search("Python memory management", final_k=5)

# Measure precision
def precision_at_k(results, relevant_ids):
    relevant_count = sum(1 for r in results if r.id in relevant_ids)
    return relevant_count / len(results)

print(f"Vector only: {precision_at_k(vector_only, ground_truth):.2%}")
# ~60% precision

print(f"With reranking: {precision_at_k(reranked, ground_truth):.2%}")
# ~85% precision (25% improvement!)

```

Latency tradeoff:

python

```

# Vector search only: 50ms
# Vector search + reranking: 200ms

# But better quality with less noise:
# - Fewer tokens in context (cost savings)
# - Better LLM answers (less distracted by irrelevant docs)

```

When to use:

- High-value queries (worth the latency)

- After initial vector search (not on entire corpus)
- When precision > speed

Case 3: Maximum Marginal Relevance (MMR)

Problem: Top-k results are often redundant (similar to each other).

Solution: MMR balances relevance with diversity.

python

```
def mmr_rerank(query_embedding, candidates, lambda_param=0.5, final_k=5):
    """
    Maximum Marginal Relevance

    Select documents that are:
    1. Relevant to query
    2. Diverse from already-selected documents

    lambda_param: tradeoff (1=relevance only, 0=diversity only)
    """
    selected = []
    remaining = candidates.copy()

    while len(selected) < final_k and remaining:
        if not selected:
            # First selection: most relevant
            scores = [
                cosine_similarity(query_embedding, c.embedding)
                for c in remaining
            ]
            best_idx = np.argmax(scores)
        else:
            # Subsequent selections: balance relevance and diversity
            mmr_scores = []

            for candidate in remaining:
                # Relevance to query
                relevance = cosine_similarity(
                    query_embedding,
                    candidate.embedding
                )

                # Max similarity to already-selected docs (want to minimize)
                max_sim_to_selected = max(
```

```

        cosine_similarity(candidate.embedding, s.embedding)
        for s in selected
    )

    # MMR score
    mmr = lambda_param * relevance - (1 - lambda_param) * max_sim_to_selected
    mmr_scores.append(mmr)

    best_idx = np.argmax(mmr_scores)

    # Add to selected, remove from remaining
    selected.append(remaining[best_idx])
    remaining.pop(best_idx)

    return selected

# Usage
candidates = vector_db.search(query, top_k=20)

# Without MMR: Top 5 might all be similar
standard_top5 = candidates[:5]
# Doc 1: "Python lists are mutable..."
# Doc 2: "Python lists can be modified..."
# Doc 3: "Lists in Python are mutable..."
# Doc 4: "You can change Python lists..."
# Doc 5: "Python list mutability means..."
# All redundant!

# With MMR: Top 5 are diverse
mmr_top5 = mmr_rerank(query_embedding, candidates, lambda_param=0.7)
# Doc 1: "Python lists are mutable..."
# Doc 2: "Python dictionaries store key-value pairs..."
# Doc 3: "Python sets contain unique elements..."
# Doc 4: "Python tuples are immutable..."
# Doc 5: "Python arrays are from numpy..."
# Diverse coverage!

```

Impact: Reduces redundancy by 60-80%, improves answer comprehensiveness.

Expert-Level Comprehension Tests

Question 1: Your RAG system returns irrelevant results 40% of the time. Users report "the bot doesn't answer my question." Walk me through your systematic debugging process.

Expected Answer

****Step 1: Isolate retrieval vs. generation**** python

```
# Test retrieval independently
query = "How do I reset my password?"
retrieved = vector_db.search(query, top_k=5)

# Manual inspection
for i, doc in enumerate(retrieved):
    print(f"{i+1}. Score: {doc.score:.3f}")
    print(f"    {doc.text[:200]}")
    print()

# Question: Are retrieved docs actually relevant?
# - If YES → generation problem
# - If NO → retrieval problem (most likely)
```

****Step 2: Measure retrieval quality**** python

```
# Get ground truth (manually label 50-100 queries)
test_cases = [
    {
        'query': 'How do I reset my password?',
        'relevant_doc_ids': [42, 108, 251]
    },
    # ... more test cases
]

# Calculate retrieval metrics
def evaluate_retrieval(test_cases, top_k=5):
    mrr_scores = []
    recall_at_k = []

    for test in test_cases:
        results = vector_db.search(test['query'], top_k=top_k)
        result_ids = [r.id for r in results]

        # Mean Reciprocal Rank
        for i, doc_id in enumerate(result_ids):
            if doc_id in test['relevant_doc_ids']:
                mrr_scores.append(1 / (i + 1))
                break
        else:
            mrr_scores.append(0)
```

```

        # Recall@k
        found = sum(1 for id in result_ids if id in test['relevant_doc_ids'])
        recall = found / len(test['relevant_doc_ids'])
        recall_at_k.append(recall)

    return {
        'MRR': np.mean(mrr_scores),
        'Recall@k': np.mean(recall_at_k)
    }

metrics = evaluate_retrieval(test_cases)
print(f"MRR: {metrics['MRR']:.3f}") # Target: >0.7
print(f"Recall@5: {metrics['Recall@k']:.3f}") # Target: >0.6

```

****Step 3: Diagnose root cause**** python

```

# Hypothesis 1: Poor chunking
# Check: Are relevant info split across chunks?
def analyze_chunking(failing_queries):
    for query in failing_queries:
        results = vector_db.search(query, top_k=10)

        # Check if combining adjacent chunks would help
        for i in range(len(results)-1):
            if results[i].source == results[i+1].source:
                combined = results[i].text + " " + results[i+1].text
                print(f"Adjacent chunks from same doc: {results[i].source}")
                print(f"Consider larger chunk size")

# Hypothesis 2: Query-document mismatch
# Check: Query embeddings vs doc embeddings
query_emb = embed(query)
doc_emb = embed(relevant_doc_text)
similarity = cosine_similarity(query_emb, doc_emb)

if similarity < 0.6:
    print("Query-document embedding mismatch")
    print("Solution: Query transformation or asymmetric embeddings")

# Hypothesis 3: Missing metadata filtering
# Check: Are wrong document types being retrieved?
retrieved_types = [r.metadata.get('type') for r in retrieved]
print(f"Document types: {set(retrieved_types)}")

```



```
if 'internal_notes' in retrieved_types:
    print("Solution: Add metadata filter to exclude internal docs")
```

****Step 4: Test fixes**** python

```
# Fix 1: Reranking
reranked = rerank_with_cross_encoder(retrieved)
new_mrr = evaluate_retrieval(test_cases, reranked)

# Fix 2: Query transformation
hyde_query = generate_hypothetical_answer(query)
results_hyde = vector_db.search(hyde_query, top_k=5)

# Fix 3: Hybrid search
results_hybrid = hybrid_search(query, alpha=0.6)

# Compare improvements
improvements = {
    'Baseline': 0.42,
    'With reranking': 0.68, # +62% MRR
    'With HyDE': 0.59, # +40% MRR
    'Hybrid search': 0.71 # +69% MRR (best)
}
```

****Anti-pattern:**** "Just increase top-k from 5 to 20" (adds noise, wastes tokens, doesn't fix root cause)

Question 2: You have 1M documents. Retrieval takes 3 seconds per query (unacceptable). How do you optimize for <500ms while maintaining quality?

Expected Answer

****Latency breakdown analysis:**** python

```
import time

def profile_retrieval(query):
    timings = {}

    start = time.time()
    # Embedding generation
    query_emb = embed(query)
    timings['embedding'] = time.time() - start

    start = time.time()
```

```

# Vector search
results = vector_db.search(query_emb, top_k=20)
timings['vector_search'] = time.time() - start

start = time.time()
# Reranking
reranked = cross_encoder.predict([[query, r.text] for r in results])
timings['reranking'] = time.time() - start

return timings

timings = profile_retrieval(test_query)
print(timings)
# {
#   'embedding': 0.05s,
#   'vector_search': 2.8s, ← Bottleneck!
#   'reranking': 0.15s
# }

```

****Optimization 1: Index tuning**** python

```

# Problem: Using Flat index (exact search,  $O(n)$ )
# Solution: Use HNSW index (approximate,  $O(\log n)$ )

# Before
collection = client.create_collection(
    name="docs",
    metadata={"hnsw:space": "cosine"} # Default HNSW params
)
# Search time: 2.8s

# After
collection = client.create_collection(
    name="docs_optimized",
    metadata={
        "hnsw:space": "cosine",
        "hnsw:construction_ef": 200, # Build quality
        "hnsw:M": 16, # Connections per node
        "hnsw:search_ef": 50 # Search quality (key param)
    }
)
# Search time: 0.3s (9x faster!)

```

```
# Tradeoff: 98% recall (vs 100% with Flat)
# Acceptable for most use cases
```

****Optimization 2: Metadata pre-filtering**** python

```
# Problem: Searching entire 1M docs
# Solution: Pre-filter by metadata

# Before
results = vector_db.search(query, top_k=20) # Searches 1M docs

# After
results = vector_db.search(
    query,
    top_k=20,
    filter={
        'document_type': 'user_manual', # Only 50k docs
        'version': 'v3'                  # Only 10k docs
    }
)
# Searches only 10k docs (100x smaller)
# Search time: 0.03s (93x faster!)
```

****Optimization 3: Separate indices by category**** python

```
# Instead of one 1M doc index
large_index = VectorDB("all_docs") # 1M docs

# Create category-specific indices
indices = {
    'user_manuals': VectorDB("user_manuals"), # 50k docs
    'api_docs': VectorDB("api_docs"),         # 30k docs
    'troubleshooting': VectorDB("troubleshooting"), # 100k docs
    'faqs': VectorDB("faqs")                  # 10k docs
}

# Route query to appropriate index
def route_query(query):
    category = classify_query(query) # Fast classifier
    return indices[category].search(query, top_k=5)

# Result: Search 10-100k docs instead of 1M
# Search time: 0.05-0.2s
```

****Optimization 4: Caching** python**

```
import hashlib
from functools import lru_cache

class CachedRetriever:
    def __init__(self, vector_db, cache_ttl=3600):
        self.vector_db = vector_db
        self.cache = {}
        self.cache_ttl = cache_ttl

    def search(self, query, top_k=5):
        # Hash query for cache key
        cache_key = hashlib.md5(query.encode()).hexdigest()

        # Check cache
        if cache_key in self.cache:
            cached_time, results = self.cache[cache_key]
            if time.time() - cached_time < self.cache_ttl:
                return results # Cache hit (instant!)

        # Cache miss - do search
        results = self.vector_db.search(query, top_k=top_k)
        self.cache[cache_key] = (time.time(), results)

        return results

# With cache: 30-40% of queries are cache hits
# Cache hit time: <1ms
```

****Combined optimization:** python**

```
# Baseline: 3000ms
# + HNSW index: 300ms (10x faster)
# + Metadata filtering: 100ms (3x faster)
# + Caching: 60ms avg (40% cache hit rate)
# Final: 60ms average (50x improvement!)

# Quality check
assert quality_after_optimization > 0.95 * baseline_quality
```

Question 3: Users report "the bot makes up information not in the docs." How do you diagnose and fix this hallucination issue in your RAG system?

Expected Answer

****Diagnosis:**** python

```
# Step 1: Measure hallucination rate
def detect_hallucination(query, retrieved_docs, answer):
    """Check if answer is grounded in retrieved docs"""

    # Extract claims from answer
    claims = extract_claims(answer)

    # Check each claim against docs
    doc_text = " ".join([d.text for d in retrieved_docs])

    hallucinated_claims = []
    for claim in claims:
        if not claim_appears_in_text(claim, doc_text):
            hallucinated_claims.append(claim)

    hallucination_rate = len(hallucinated_claims) / len(claims)

    return {
        'hallucinated': hallucination_rate > 0.2,
        'rate': hallucination_rate,
        'claims': hallucinated_claims
    }

# Run on 100 test cases
hallucination_cases = []
for test in test_cases:
    retrieved = vector_db.search(test['query'], top_k=5)
    answer = generate_answer(test['query'], retrieved)

    result = detect_hallucination(test['query'], retrieved, answer)
    if result['hallucinated']:
        hallucination_cases.append((test, result))

print(f"Hallucination rate: {len(hallucination_cases)}%")
# If >10%, major problem
```

****Root Cause Analysis:**** python

```
# Hypothesis 1: Retrieved docs are irrelevant
# LLM hallucinates because it has no good information
```

```

for test, result in hallucination_cases:
    retrieved = vector_db.search(test['query'], top_k=5)

    # Check relevance
    avg_score = np.mean([r.score for r in retrieved])

    if avg_score < 0.5:
        print(f"Low relevance retrieval: {avg_score:.3f}")
        print("Solution: Improve retrieval quality")

# Hypothesis 2: Context is too long, LLM gets confused
# Check context length
context_lengths = []
for test in hallucination_cases:
    retrieved = vector_db.search(test['query'], top_k=5)
    context = "\n".join([r.text for r in retrieved])
    context_lengths.append(len(context.split()))

avg_context = np.mean(context_lengths)
if avg_context > 2000:
    print(f"Context too long: {avg_context} words")
    print("Solution: Reduce top-k or summarize chunks")

# Hypothesis 3: Prompt doesn't enforce groundedness
# Check prompt
if "only use the provided context" not in system_prompt:
    print("Solution: Add grounding instructions to prompt")

```

****Fixes:**** python

```

# Fix 1: Improve prompt to enforce groundedness
GROUNDED_PROMPT = """
You are a helpful assistant that answers questions using ONLY the provided context.

CRITICAL RULES:
1. If the answer is not in the context, say "I don't have enough information"
2. NEVER make up information
3. Quote directly from the context when possible
4. If you're unsure, say so

Context:
{context}

Question: {query}

```

Answer:

"""

Fix 2: Add citation requirement

CITATION_PROMPT = """

Answer the question using the provided context. For each claim you make, cite the source.

Documents:

[1] {doc1}

[2] {doc2}

...

Question: {query}

Answer with citations:

"""

Fix 3: Two-step generation (retrieve → verify)

def grounded_generation(query, retrieved_docs):

Step 1: Generate answer

answer = llm.complete(f"{context}\nQ: {query}\nA:")

Step 2: Verify each claim

claims = extract_claims(answer)

verified_claims = []

for claim in claims:

verification_prompt = f"""

Context: {context}

Claim: {claim}

Is this claim supported by the context? Answer yes or no.

If yes, quote the supporting text.

"""

verification = llm.complete(verification_prompt)

if "yes" in verification.lower():

verified_claims.append(claim)

Reconstruct answer from verified claims only

grounded_answer = construct_answer(verified_claims)

return grounded_answer

Fix 4: Use higher-quality retrieval

```
# Better retrieval = less hallucination
reranked = rerank_with_cross_encoder(retrieved, top_k=3)
# Use only top-3 most relevant (high precision)

# Measure improvement
baseline_hallucination = 0.24 # 24%
with_grounded_prompt = 0.18 # 18% (-25%)
with_citations = 0.12 # 12% (-50%)
with_verification = 0.06 # 6% (-75%)
```

****Long-term solution: Fine-tune on grounded examples**** python

```
# Create training data of grounded responses
training_examples = [
    {
        'context': '...',
        'query': '...',
        'good_answer': '...', # Fully grounded
        'bad_answer': '...' # Contains hallucination
    }
]

# Fine-tune model to prefer grounded responses
# (Advanced - beyond scope here, but worth mentioning)
```

3. Fastest Path to Competence (Execution-Focused)

Prerequisites (Week 0: Setup)

Install:

bash

```
pip install \
    chromadb \ # Vector database
    sentence-transformers \ # Embeddings
    rank-bm25 \ # Keyword search
    openai anthropic \ # LLM APIs
    pandas numpy \ # Data manipulation
    scikit-learn \ # Metrics
```



```
pytest \           # Testing
datasets          # Evaluation datasets
```

Repository structure:

```
production-rag/
├── data/
│   ├── documents/      # Source documents
│   ├── chunks/        # Processed chunks
│   └── eval/           # Test cases with ground truth
├── src/
│   ├── retrieval/
│   │   ├── vector.py   # Vector search
│   │   ├── hybrid.py   # Vector + keyword
│   │   ├── reranker.py # Cross-encoder reranking
│   │   └── mmr.py      # MMR diversity
│   ├── generation/
│   │   ├── prompts.py  # RAG prompts
│   │   └── grounding.py # Hallucination prevention
│   ├── evaluation/
│   │   ├── retrieval_metrics.py # MRR, NDCG, Recall
│   │   └── e2e_metrics.py      # Answer quality
│   └── diagnosis/
│       └── failure_analysis.py
├── tests/
│   └── test_rag_pipeline.py
└── README.md
```

Week 1: Retrieval Fundamentals & Failure Diagnosis

Learning Objectives:

- Build baseline RAG system
- Implement retrieval metrics
- Create systematic failure taxonomy
- Debug retrieval failures

Day 1-2: Baseline RAG Implementation

Hands-on Lab:

python

```

# src/retrieval/vector.py
from sentence_transformers import SentenceTransformer
import chromadb
from typing import List, Dict, Any

class VectorRetriever:
    def __init__(self, model_name='all-MiniLM-L6-v2'):
        self.embedding_model = SentenceTransformer(model_name)

        # Initialize vector DB
        self.client = chromadb.Client()
        self.collection = self.client.create_collection(
            name="documents",
            metadata={"hnsw:space": "cosine"}
        )

    def add_documents(self, documents: List[Dict[str, Any]]):
        """Add documents to vector database"""
        # Extract texts
        texts = [doc['text'] for doc in documents]

        # Generate embeddings
        embeddings = self.embedding_model.encode(
            texts,
            show_progress_bar=True,
            batch_size=32
        )

        # Add to vector DB
        self.collection.add(
            ids=[str(i) for i in range(len(documents))],
            embeddings=embeddings.tolist(),
            documents=texts,
            metadatas=[{
                'source': doc.get('source', ''),
                'title': doc.get('title', '')
            } for doc in documents]
        )

    def search(self, query: str, top_k: int = 5):
        """Search for relevant documents"""
        # Embed query
        query_embedding = self.embedding_model.encode([query])[0]

        # Search

```

```

        results = self.collection.query(
            query_embeddings=[query_embedding.tolist()],
            n_results=top_k
        )

    # Format results
    retrieved = []
    for i in range(len(results['ids'][0])):
        retrieved.append({
            'id': results['ids'][0][i],
            'text': results['documents'][0][i],
            'score': 1 - results['distances'][0][i], # Convert distance to similarity
            'metadata': results['metadatas'][0][i]
        })

    return retrieved

# src/generation/prompts.py
RAG_SYSTEM_PROMPT = """You are a helpful assistant that answers questions based on the p

IMPORTANT RULES:
1. Base your answer ONLY on the provided context
2. If the context doesn't contain the answer, say "I don't have enough information"
3. Quote relevant parts of the context when appropriate
4. Be concise and direct

Context:
{context}

Question: {query}

Answer: """

def generate_answer(query: str, retrieved_docs: List[Dict], llm_client):
    """Generate answer from retrieved documents"""
    # Format context
    context = "\n\n".join([
        f"Document {i+1}: {doc['text']}"
        for i, doc in enumerate(retrieved_docs)
    ])

    # Generate
    prompt = RAG_SYSTEM_PROMPT.format(context=context, query=query)

    response = llm_client.chat.completions.create(
        model="gpt-3.5-turbo",

```

```

        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )

    return response.choices[0].message.content

# End-to-end pipeline
class RAGPipeline:
    def __init__(self, retriever, llm_client):
        self.retriever = retriever
        self.llm_client = llm_client

    def query(self, question: str, top_k: int = 5):
        # Retrieve
        retrieved = self.retriever.search(question, top_k=top_k)

        # Generate
        answer = generate_answer(question, retrieved, self.llm_client)

        return {
            'answer': answer,
            'retrieved_docs': retrieved
        }

# Usage
from openai import OpenAI

# Initialize
retriever = VectorRetriever()
retriever.add_documents(documents)

llm_client = OpenAI()
rag = RAGPipeline(retriever, llm_client)

# Query
result = rag.query("How do I reset my password?")
print(result['answer'])

```

Deliverable: Working baseline RAG with:

- Vector retrieval
- Answer generation
- Document tracking

Day 3-5: Retrieval Metrics & Evaluation

Hands-on Lab:

python

```
# src/evaluation/retrieval_metrics.py
import numpy as np
from typing import List, Set

class RetrievalEvaluator:
    def __init__(self):
        self.results = []

    def evaluate_query(self,
                      retrieved_ids: List[str],
                      relevant_ids: Set[str],
                      k: int = 5):
        """Evaluate single query"""
        retrieved_at_k = retrieved_ids[:k]

        # Precision@k
        relevant_retrieved = set(retrieved_at_k) & relevant_ids
        precision = len(relevant_retrieved) / k if k > 0 else 0

        # Recall@k
        recall = len(relevant_retrieved) / len(relevant_ids) if relevant_ids else 0

        # Mean Reciprocal Rank (MRR)
        reciprocal_rank = 0
        for i, doc_id in enumerate(retrieved_at_k):
            if doc_id in relevant_ids:
                reciprocal_rank = 1 / (i + 1)
                break

        # Average Precision (AP)
        # Average of precision at each relevant doc position
        precisions_at_relevant = []
        num_relevant_seen = 0

        for i, doc_id in enumerate(retrieved_at_k):
            if doc_id in relevant_ids:
                num_relevant_seen += 1
                precision_at_i = num_relevant_seen / (i + 1)
                precisions_at_relevant.append(precision_at_i)
```

```

        ap = np.mean(precisions_at_relevant) if precisions_at_relevant else 0

    # NDCG@k (Normalized Discounted Cumulative Gain)
    dcg = sum(
        (1 if retrieved_at_k[i] in relevant_ids else 0) / np.log2(i + 2)
        for i in range(k)
    )

    # Ideal DCG (all relevant docs at top)
    idcg = sum(
        1 / np.log2(i + 2)
        for i in range(min(k, len(relevant_ids)))
    )

    ndcg = dcg / idcg if idcg > 0 else 0

    metrics = {
        'precision@k': precision,
        'recall@k': recall,
        'mrr': reciprocal_rank,
        'ap': ap,
        'ndcg@k': ndcg
    }

    self.results.append(metrics)
    return metrics

def get_aggregate_metrics(self):
    """Aggregate metrics across all queries"""
    if not self.results:
        return {}

    aggregated = {}
    for metric_name in self.results[0].keys():
        values = [r[metric_name] for r in self.results]
        aggregated[metric_name] = {
            'mean': np.mean(values),
            'std': np.std(values),
            'min': np.min(values),
            'max': np.max(values)
        }

    # MAP (Mean Average Precision)
    aggregated['MAP'] = np.mean([r['ap'] for r in self.results])

    # MRR (Mean Reciprocal Rank)

```

```

        aggregated['MRR'] = np.mean([r['mrr'] for r in self.results])

    return aggregated

# Create evaluation dataset
eval_dataset = [
    {
        'query': 'How do I reset my password?',
        'relevant_doc_ids': {'doc_42', 'doc_108', 'doc_251'}
    },
    {
        'query': 'What are the system requirements?',
        'relevant_doc_ids': {'doc_15', 'doc_89'}
    },
    # ... 50-100 more test cases
]

# Evaluate
evaluator = RetrievalEvaluator()

for test_case in eval_dataset:
    retrieved = retriever.search(test_case['query'], top_k=10)
    retrieved_ids = [doc['id'] for doc in retrieved]

    metrics = evaluator.evaluate_query(
        retrieved_ids,
        test_case['relevant_doc_ids'],
        k=5
    )

    print(f"Query: {test_case['query'][:50]}...")
    print(f" Precision@5: {metrics['precision@k']:.3f}")
    print(f" Recall@5: {metrics['recall@k']:.3f}")
    print(f" MRR: {metrics['mrr']:.3f}")

# Aggregate results
aggregate = evaluator.get_aggregate_metrics()
print("\nAggregate Metrics:")
print(f"MAP: {aggregate['MAP']:.3f}")
print(f"MRR: {aggregate['MRR']:.3f}")
print(f"Precision@5: {aggregate['precision@k']['mean']:.3f} ± {aggregate['precision@k']['std']:.3f}")
print(f"NDCG@5: {aggregate['ndcg@k']['mean']:.3f} ± {aggregate['ndcg@k']['std']:.3f}")

```

Deliverable: Evaluation framework with:

- 50+ labeled test cases
- Retrieval metrics (MRR, NDCG, MAP)
- Baseline performance numbers

Target metrics:

- MRR > 0.6
- Precision@5 > 0.7
- NDCG@5 > 0.65

Day 6-7: Failure Taxonomy & Diagnosis

Hands-on Lab:

python

```
# src/diagnosis/failure_analysis.py
from enum import Enum
from typing import List, Dict, Optional

class FailureMode(Enum):
    NO_RESULTS = "no_results_retrieved"
    LOW_RELEVANCE = "low_relevance_scores"
    MISSING_INFO = "missing_key_information"
    NOISY_RESULTS = "too_much_irrelevant_content"
    QUERY_MISMATCH = "query_document_mismatch"
    CHUNKING_ISSUE = "information_split_across_chunks"
    HALLUCINATION = "answer_not_grounded"

class RAGDiagnostics:
    def __init__(self, retriever, llm_client):
        self.retriever = retriever
        self.llm_client = llm_client

    def diagnose(self, query: str, expected_answer: Optional[str] = None) -> Dict:
        """Comprehensive diagnostic analysis"""
        diagnosis = {
            'query': query,
            'issues': [],
            'recommendations': []
        }

        # Step 1: Retrieve documents
```



```

retrieved = self.retriever.search(query, top_k=10)

# Check for no results
if not retrieved:
    diagnosis['issues'].append({
        'type': FailureMode.NO_RESULTS,
        'severity': 'critical',
        'description': 'No documents retrieved'
    })
    diagnosis['recommendations'].append(
        'Check if documents are indexed correctly'
    )
    return diagnosis

# Check relevance scores
top_score = retrieved[0]['score']
avg_score = np.mean([d['score'] for d in retrieved[:5]])

if top_score < 0.5:
    diagnosis['issues'].append({
        'type': FailureMode.LOW_RELEVANCE,
        'severity': 'high',
        'description': f'Top result score is low: {top_score:.3f}',
        'data': {'top_score': top_score}
    })
    diagnosis['recommendations'].extend([
        'Consider query transformation (HyDE)',
        'Check if embedding model is appropriate',
        'Try hybrid search (vector + keyword)'
    ])

# Check for information spread across chunks
sources = [d['metadata'].get('source', '') for d in retrieved[:5]]
if len(set(sources)) < len(sources) * 0.3: # Many from same source
    diagnosis['issues'].append({
        'type': FailureMode.CHUNKING_ISSUE,
        'severity': 'medium',
        'description': 'Multiple chunks from same document in top results',
        'data': {'sources': sources}
    })
    diagnosis['recommendations'].append(
        'Consider larger chunk size or post-retrieval merging'
    )

# Check for query-document mismatch
query_emb = self.retriever.embedding_model.encode([query])[0]

```

```

doc_emb = self.retriever.embedding_model.encode([retrieved[0]['text']])[0]

from numpy.linalg import norm
sim = np.dot(query_emb, doc_emb) / (norm(query_emb) * norm(doc_emb))

if sim < 0.6 and top_score > 0.6:
    # Vector DB says high similarity but direct comparison is low
    diagnosis['issues'].append({
        'type': FailureMode.QUERY_MISMATCH,
        'severity': 'medium',
        'description': 'Query and document embeddings have low direct similarity'
    })
    diagnosis['recommendations'].append(
        'Use asymmetric embeddings or query transformation'
    )

# Generate answer and check grounding
answer = self.generate_answer(query, retrieved[:5])

if not self.is_answer_grounded(answer, retrieved[:5]):
    diagnosis['issues'].append({
        'type': FailureMode.HALLUCINATION,
        'severity': 'critical',
        'description': 'Generated answer contains information not in retrieved d'
    })
    diagnosis['recommendations'].extend([
        'Strengthen grounding instructions in prompt',
        'Add citation requirements',
        'Implement answer verification step'
    ])

# Check if expected answer is in retrieved docs
if expected_answer:
    found = any(
        self.text_contains_info(d['text'], expected_answer)
        for d in retrieved[:5]
    )

    if not found:
        diagnosis['issues'].append({
            'type': FailureMode.MISSING_INFO,
            'severity': 'high',
            'description': 'Expected information not in top-5 results'
        })
        diagnosis['recommendations'].extend([
            'Increase top-k',

```

```

        'Improve chunking strategy',
        'Add reranking step'
    ])

    return diagnosis

def is_answer_grounded(self, answer: str, docs: List[Dict]) -> bool:
    """Check if answer is grounded in documents"""
    # Extract key claims from answer
    claims = self.extract_claims(answer)

    # Check each claim
    doc_text = " ".join([d['text'] for d in docs])

    grounded_count = sum(
        1 for claim in claims
        if claim.lower() in doc_text.lower()
    )

    return grounded_count / len(claims) > 0.8 if claims else True

def extract_claims(self, text: str) -> List[str]:
    """Extract factual claims from text"""
    # Simple sentence splitting (production would use more sophisticated NLP)
    sentences = text.split('.')
    return [s.strip() for s in sentences if len(s.strip()) > 10]

def text_contains_info(self, text: str, info: str) -> bool:
    """Check if text contains information (fuzzy match)"""
    # Production would use semantic similarity
    return info.lower() in text.lower()

def generate_diagnostic_report(self, diagnosis: Dict) -> str:
    """Generate human-readable report"""
    report = [f"Diagnostic Report for Query: '{diagnosis['query']}' ", "=" * 60]

    if not diagnosis['issues']:
        report.append("\n✓ No issues detected")
    else:
        report.append(f"\n⚠ Found {len(diagnosis['issues'])} issues:\n")

        for i, issue in enumerate(diagnosis['issues'], 1):
            report.append(f"{i}. [{issue['severity'].upper()}] {issue['type'].value}")
            report.append(f"    {issue['description']}")
            if 'data' in issue:
                report.append(f"    Data: {issue['data']}")

```

```

        report.append("")

    if diagnosis['recommendations']:
        report.append("\nRecommendations:")
        for i, rec in enumerate(diagnosis['recommendations'], 1):
            report.append(f"{i}. {rec}")

    return "\n".join(report)

# Usage
diagnostics = RAGDiagnostics(retriever, llm_client)

# Diagnose failing query
failing_query = "How do I configure SSL?"
expected = "To configure SSL, edit the config.yaml file..."

diagnosis = diagnostics.diagnose(failing_query, expected_answer=expected)
print(diagnostics.generate_diagnostic_report(diagnosis))

```

Deliverable: Diagnostic framework that:

- Identifies failure modes
- Provides specific recommendations
- Generates actionable reports

Week 2: Advanced Retrieval Techniques

Learning Objectives:

- Implement hybrid search
- Add cross-encoder reranking
- Apply MMR for diversity
- Measure improvements

Day 8-10: Hybrid Search Implementation

Hands-on Lab:

python

```

# src/retrieval/hybrid.py
from rank_bm25 import BM25Okapi

```

```

import numpy as np
from typing import List, Dict

class HybridRetriever:
    def __init__(self, vector_retriever, documents):
        self.vector_retriever = vector_retriever
        self.documents = documents

        # Build BM25 index
        tokenized = [doc['text'].split() for doc in documents]
        self.bm25 = BM25Okapi(tokenized)

        # Create ID mapping
        self.doc_id_to_idx = {doc['id']: i for i, doc in enumerate(documents)}

    def search(self, query: str, top_k: int = 5, alpha: float = 0.5):
        """
        Hybrid search with configurable weighting

        alpha: Weight for vector search (0-1)
            1.0 = pure vector
            0.0 = pure keyword
            0.5 = balanced
        """
        # Vector search
        vector_results = self.vector_retriever.search(query, top_k=top_k*3)
        vector_scores = {r['id']: r['score'] for r in vector_results}

        # Keyword search
        tokenized_query = query.split()
        bm25_scores = self.bm25.get_scores(tokenized_query)

        # Normalize scores to [0, 1]
        def normalize(scores):
            min_s, max_s = min(scores), max(scores)
            if max_s == min_s:
                return [1.0] * len(scores)
            return [(s - min_s) / (max_s - min_s) for s in scores]

        # Get all document IDs
        all_doc_ids = set(vector_scores.keys()) | set(range(len(self.documents)))

        # Combine scores
        combined = {}
        for doc_id in all_doc_ids:
            if isinstance(doc_id, str):

```

```

        idx = self.doc_id_to_idx.get(doc_id, 0)
    else:
        idx = doc_id
        doc_id = self.documents[idx]['id']

    v_score = vector_scores.get(doc_id, 0)
    k_score = bm25_scores[idx] if idx < len(bm25_scores) else 0

    # Normalize keyword score
    k_score_norm = k_score / (max(bm25_scores) if max(bm25_scores) > 0 else 1)

    # Combined score
    combined[doc_id] = alpha * v_score + (1 - alpha) * k_score_norm

# Rank by combined score
ranked = sorted(combined.items(), key=lambda x: x[1], reverse=True)[:top_k]

# Return documents
results = []
for doc_id, score in ranked:
    idx = self.doc_id_to_idx[doc_id]
    results.append({
        'id': doc_id,
        'text': self.documents[idx]['text'],
        'score': score,
        'metadata': self.documents[idx].get('metadata', {})
    })

return results

# Evaluation: Compare vector-only vs hybrid
evaluator_vector = RetrievalEvaluator()
evaluator_hybrid = RetrievalEvaluator()

for test_case in eval_dataset:
    # Vector only
    vector_results = vector_retriever.search(test_case['query'], top_k=5)
    vector_ids = [r['id'] for r in vector_results]
    evaluator_vector.evaluate_query(vector_ids, test_case['relevant_doc_ids'])

    # Hybrid
    hybrid_results = hybrid_retriever.search(test_case['query'], top_k=5, alpha=0.6)
    hybrid_ids = [r['id'] for r in hybrid_results]
    evaluator_hybrid.evaluate_query(hybrid_ids, test_case['relevant_doc_ids'])

print("Vector Only:")

```

```

print(f"  MRR: {evaluator_vector.get_aggregate_metrics()['MRR']:.3f}")
print(f"  Precision@5: {evaluator_vector.get_aggregate_metrics()['precision@k'][:5]['mean']:.3f}")

print("\nHybrid (alpha=0.6):")
print(f"  MRR: {evaluator_hybrid.get_aggregate_metrics()['MRR']:.3f}")
print(f"  Precision@5: {evaluator_hybrid.get_aggregate_metrics()['precision@k'][:5]['mean']:.3f}")

# Expected improvement: 10-20% better MRR

```

Deliverable: Hybrid retriever with:

- Configurable vector/keyword weighting
- Measured improvement over baseline
- Alpha parameter tuning

Day 11-13: Cross-Encoder Reranking

Hands-on Lab:

python

```

# src/retrieval/reranker.py
from sentence_transformers import CrossEncoder

class CrossEncoderReranker:
    def __init__(self, model_name='cross-encoder/ms-marco-MiniLM-L-6-v2'):
        self.model = CrossEncoder(model_name)

    def rerank(self, query: str, documents: List[Dict], top_k: int = 5):
        """Rerank documents using cross-encoder"""
        # Prepare pairs
        pairs = [[query, doc['text']] for doc in documents]

        # Get reranking scores
        scores = self.model.predict(pairs)

        # Combine with documents
        scored_docs = [
            {**doc, 'rerank_score': float(score)}
            for doc, score in zip(documents, scores)
        ]

        # Sort by rerank score
        scored_docs.sort(key=lambda x: x['rerank_score'], reverse=True)

```

```

        return scored_docs[:top_k]

class TwoStageRetriever:
    def __init__(self, retriever, reranker):
        self.retriever = retriever
        self.reranker = reranker

    def search(self, query: str, initial_k: int = 20, final_k: int = 5):
        """Two-stage retrieval"""
        # Stage 1: Fast retrieval (recall-focused)
        candidates = self.retriever.search(query, top_k=initial_k)

        if not candidates:
            return []

        # Stage 2: Slow reranking (precision-focused)
        reranked = self.reranker.rerank(query, candidates, top_k=final_k)

        return reranked

# Benchmark
import time

reranker = CrossEncoderReranker()
two_stage = TwoStageRetriever(vector_retriever, reranker)

evaluator_baseline = RetrievalEvaluator()
evaluator_reranked = RetrievalEvaluator()

latencies_baseline = []
latencies_reranked = []

for test_case in eval_dataset:
    # Baseline
    start = time.time()
    baseline_results = vector_retriever.search(test_case['query'], top_k=5)
    latencies_baseline.append(time.time() - start)

    baseline_ids = [r['id'] for r in baseline_results]
    evaluator_baseline.evaluate_query(baseline_ids, test_case['relevant_doc_ids'])

    # Reranked
    start = time.time()
    reranked_results = two_stage.search(test_case['query'], initial_k=20, final_k=5)
    latencies_reranked.append(time.time() - start)

```



```

reranked_ids = [r['id'] for r in reranked_results]
evaluator_reranked.evaluate_query(reranked_ids, test_case['relevant_doc_ids'])

print("Baseline:")
print(f"  MRR: {evaluator_baseline.get_aggregate_metrics()['MRR']:.3f}")
print(f"  Precision@5: {evaluator_baseline.get_aggregate_metrics()['precision@k']['mean@5']:.3f}")
print(f"  Avg Latency: {np.mean(latencies_baseline)*1000:.0f}ms")

print("\nWith Reranking:")
print(f"  MRR: {evaluator_reranked.get_aggregate_metrics()['MRR']:.3f}")
print(f"  Precision@5: {evaluator_reranked.get_aggregate_metrics()['precision@k']['mean@5']:.3f}")
print(f"  Avg Latency: {np.mean(latencies_reranked)*1000:.0f}ms")

# Expected:
# - 20-30% improvement in MRR
# - 3-4x latency increase (but worth it for quality)

```

Deliverable: Reranking system with:

- Two-stage retrieval
- Quality vs latency analysis
- Measured precision improvement

Day 14: MMR Diversity

Hands-on Lab:

python

```

# src/retrieval/mmr.py
def mmr_rerank(query_embedding, candidates, lambda_param=0.5, k=5):
    """Maximum Marginal Relevance for diversity"""
    selected = []
    remaining = candidates.copy()

    while len(selected) < k and remaining:
        if not selected:
            # First: most relevant
            scores = [
                np.dot(query_embedding, c['embedding'])
                for c in remaining
            ]
            best_idx = np.argmax(scores)
        else:
            # Balance relevance and diversity

```

```

mmr_scores = []

for candidate in remaining:
    # Relevance
    relevance = np.dot(query_embedding, candidate['embedding'])

    # Max similarity to selected (want LOW)
    max_sim = max(
        np.dot(candidate['embedding'], s['embedding'])
        for s in selected
    )

    # MMR
    mmr = lambda_param * relevance - (1 - lambda_param) * max_sim
    mmr_scores.append(mmr)

best_idx = np.argmax(mmr_scores)

selected.append(remaining.pop(best_idx))

return selected

# Test: Check diversity improvement
query = "Python programming basics"
results_standard = retriever.search(query, top_k=5)
results_mmr = mmr_rerank(query_embedding, candidates, lambda_param=0.7, k=5)

# Measure diversity (avg pairwise similarity)
def measure_diversity(results):
    similarities = []
    for i in range(len(results)):
        for j in range(i+1, len(results)):
            sim = np.dot(results[i]['embedding'], results[j]['embedding'])
            similarities.append(sim)
    return 1 - np.mean(similarities) # Higher = more diverse

print(f"Standard diversity: {measure_diversity(results_standard):.3f}")
print(f"MMR diversity: {measure_diversity(results_mmr):.3f}")
# MMR should be 20-40% more diverse

```

Deliverable: MMR implementation showing:

- Diversity improvement
- Relevance preservation
- Lambda parameter tuning

Week 3: Production Readiness & Portfolio

Learning Objectives:

- Build end-to-end RAG evaluation
- Implement query transformation
- Create production monitoring
- Document portfolio

Day 15-17: End-to-End Evaluation

Hands-on Lab:

python

```
# src/evaluation/e2e_metrics.py
from typing import List, Dict

class E2EEvaluator:
    def __init__(self, rag_pipeline, llm_judge):
        self.rag = rag_pipeline
        self.llm_judge = llm_judge

    def evaluate_answer_quality(self, query: str, answer: str, ground_truth: str, retrieved_docs: List[str]) >
        """Evaluate answer quality on multiple dimensions"""

        # 1. Faithfulness (is answer grounded in retrieved docs?)
        faithfulness = self.measure_faithfulness(answer, retrieved_docs)

        # 2. Relevance (does answer address the query?)
        relevance = self.measure_relevance(query, answer)

        # 3. Correctness (compared to ground truth)
        correctness = self.measure_correctness(answer, ground_truth)

        # 4. Completeness (covers all key points?)
        completeness = self.measure_completeness(answer, ground_truth)

        return {
            'faithfulness': faithfulness,
            'relevance': relevance,
            'correctness': correctness,
            'completeness': completeness,
```

```

        'overall': np.mean([faithfulness, relevance, correctness, completeness])
    }

def measure_faithfulness(self, answer: str, docs: List[Dict]) -> float:
    """Check if answer is grounded in docs (using LLM-as-judge)"""
    context = "\n\n".join([d['text'] for d in docs])

    prompt = f"""
    Context: {context}

    Answer: {answer}

    Is the answer fully supported by the context? Rate from 0-1:
    - 1.0: All claims are directly supported
    - 0.5: Some claims are supported, some are not
    - 0.0: Answer contradicts or is unrelated to context

    Respond with ONLY a number between 0 and 1.
    """

    response = self.llm_judge.complete(prompt)
    try:
        score = float(response.strip())
        return max(0.0, min(1.0, score))
    except:
        return 0.5

def measure_relevance(self, query: str, answer: str) -> float:
    """Check if answer addresses the query"""
    prompt = f"""
    Question: {query}

    Answer: {answer}

    How well does the answer address the question? Rate from 0-1:
    - 1.0: Directly and completely answers the question
    - 0.5: Partially addresses the question
    - 0.0: Does not address the question

    Respond with ONLY a number between 0 and 1.
    """

    response = self.llm_judge.complete(prompt)
    try:
        return float(response.strip())
    except:

```

```

        return 0.5

def measure_correctness(self, answer: str, ground_truth: str) -> float:
    """Compare answer to ground truth"""
    from sklearn.metrics.pairwise import cosine_similarity
    from sentence_transformers import SentenceTransformer

    model = SentenceTransformer('all-MiniLM-L6-v2')

    emb1 = model.encode([answer])[0]
    emb2 = model.encode([ground_truth])[0]

    similarity = cosine_similarity([emb1], [emb2])[0][0]
    return float(similarity)

def measure_completeness(self, answer: str, ground_truth: str) -> float:
    """Check if answer covers key points from ground truth"""
    # Extract key points from ground truth (simple: sentences)
    key_points = [s.strip() for s in ground_truth.split('.') if len(s.strip()) > 10]

    # Check how many key points are covered in answer
    covered = sum(
        1 for point in key_points
        if any(word in answer.lower() for word in point.lower().split()[:3])
    )

    return covered / len(key_points) if key_points else 1.0

# Run comprehensive evaluation
e2e_eval = E2EEvaluator(rag_pipeline, llm_client)

results = []
for test_case in eval_dataset:
    # Get RAG response
    rag_response = rag_pipeline.query(test_case['query'])

    # Evaluate
    scores = e2e_eval.evaluate_answer_quality(
        query=test_case['query'],
        answer=rag_response['answer'],
        ground_truth=test_case['expected_answer'],
        retrieved_docs=rag_response['retrieved_docs']
    )

    results.append({
        'query': test_case['query'],

```

```

        **scores
    })

# Aggregate
df = pd.DataFrame(results)
print(df[['faithfulness', 'relevance', 'correctness', 'completeness', 'overall']].describe())

```

Deliverable: E2E evaluation showing:

- Answer quality metrics
- Comparison to baseline
- Failure case analysis

Day 18-19: Production Monitoring

Hands-on Lab:

python

```

# src/monitoring/rag_monitor.py
import time
from collections import deque, defaultdict

class RAGMonitor:
    def __init__(self, window_size=100):
        self.window_size = window_size

        # Sliding windows for metrics
        self.retrieval_scores = deque(maxlen=window_size)
        self.answer_latencies = deque(maxlen=window_size)
        self.retrieval_latencies = deque(maxlen=window_size)

        # Counters
        self.total_queries = 0
        self.failed_retrievals = 0
        self.low_quality_answers = 0

        # Failure patterns
        self.failure_modes = defaultdict(int)

    def log_query(self, query: str, retrieved: List[Dict], answer: str, latency: Dict):
        """Log query metrics"""
        self.total_queries += 1

        # Retrieval quality

```

```

        if retrieved:
            top_score = retrieved[0]['score']
            self.retrieval_scores.append(top_score)

            if top_score < 0.5:
                self.low_quality_answers += 1
                self.failure_modes['low_relevance'] += 1
        else:
            self.failed_retrievals += 1
            self.failure_modes['no_results'] += 1

        # Latency
        self.answer_latencies.append(latency['total'])
        self.retrieval_latencies.append(latency['retrieval'])

        # Alert on anomalies
        self.check_alerts()

    def check_alerts(self):
        """Check for concerning patterns"""
        # Alert if retrieval quality dropping
        if len(self.retrieval_scores) == self.window_size:
            recent_avg = np.mean(list(self.retrieval_scores)[-20:])
            overall_avg = np.mean(self.retrieval_scores)

            if recent_avg < overall_avg * 0.8:
                logger.warning(
                    f"Retrieval quality drop: {recent_avg:.3f} vs {overall_avg:.3f}"
                )

        # Alert if latency spiking
        if len(self.answer_latencies) > 20:
            recent_p95 = np.percentile(list(self.answer_latencies)[-20:], 95)
            if recent_p95 > 5000: # 5 seconds
                logger.warning(f"High latency: p95 = {recent_p95:.0f}ms")

    def get_health_metrics(self):
        """Get current health status"""
        return {
            'total_queries': self.total_queries,
            'success_rate': 1 - (self.failed_retrievals / self.total_queries) if self.total_queries > 0 else 1,
            'avg_relevance': np.mean(self.retrieval_scores) if self.retrieval_scores else 0,
            'p50_latency': np.percentile(self.answer_latencies, 50) if self.answer_latencies else 0,
            'p95_latency': np.percentile(self.answer_latencies, 95) if self.answer_latencies else 0,
            'failure_modes': dict(self.failure_modes)
        }

```

```

def generate_dashboard(self):
    """Generate monitoring dashboard"""
    metrics = self.get_health_metrics()

    dashboard = f"""
RAG System Health Dashboard
{'='*60}

Query Volume: {metrics['total_queries']}
Success Rate: {metrics['success_rate']:.1%}

Retrieval Quality:
    Avg Relevance Score: {metrics['avg_relevance']:.3f}
    Target: >0.6

Latency:
    P50: {metrics['p50_latency']:.0f}ms
    P95: {metrics['p95_latency']:.0f}ms
    Target: P95 <2000ms

Failure Modes:
    """

    for mode, count in metrics['failure_modes'].items():
        dashboard += f"    {mode}: {count} ({count/metrics['total_queries']*100:.1f}%)"

    return dashboard

# Usage
monitor = RAGMonitor()

# Wrap RAG pipeline with monitoring
class MonitoredRAG:
    def __init__(self, rag_pipeline, monitor):
        self.rag = rag_pipeline
        self.monitor = monitor

    def query(self, question: str):
        start = time.time()

        # Retrieve
        retrieval_start = time.time()
        retrieved = self.rag.retriever.search(question)
        retrieval_time = (time.time() - retrieval_start) * 1000

        # Generate

```



```

        answer = self.rag.generate_answer(question, retrieved)
        total_time = (time.time() - start) * 1000

        # Log
        self.monitor.log_query(
            query=question,
            retrieved=retrieved,
            answer=answer,
            latency={'retrieval': retrieval_time, 'total': total_time}
        )

        return {'answer': answer, 'retrieved_docs': retrieved}

# Deploy with monitoring
monitored_rag = MonitoredRAG(rag_pipeline, monitor)

# Periodic reporting
import schedule

def print_dashboard():
    print(monitor.generate_dashboard())

schedule.every(1).hours.do(print_dashboard)

```

Deliverable: Monitoring system with:

- Real-time metrics tracking
- Automated alerting
- Health dashboard

Day 20-21: Portfolio Documentation

Create comprehensive README:

markdown

```

# Production-Grade RAG System

## Overview
End-to-end Retrieval-Augmented Generation system with systematic failure diagnosis, achieving high accuracy and reliability.

## Architecture

```

Query → Hybrid Retrieval → Reranking → Answer Generation

↓↓↓

Vector + BM25 Cross-Encoder Grounded Prompts

Results

Retrieval Performance

Metric	Baseline (Vector Only)	+ Hybrid Search	+ Reranking	Final
MRR	0.58	0.69 (+19%)	0.81 (+40%)	0.81
Precision@5	0.62	0.73 (+18%)	0.86 (+39%)	0.86
Recall@10	0.71	0.83 (+17%)	0.88 (+24%)	0.88

Answer Quality (E2E)

- Faithfulness: 0.92 (92% grounded in docs)
- Relevance: 0.89 (89% address query)
- Correctness: 0.87 (vs ground truth)
- Overall: 0.91/1.0

Performance

- P50 Latency: 450ms
- P95 Latency: 1200ms
- Cost: \$0.003/query
- Uptime: 99.5%

Features

1. Hybrid Retrieval

Combines vector (semantic) and keyword (BM25) search:

```
python
results = hybrid_retriever.search(
    "SKU-12345 installation guide",
    alpha=0.4 # 40% vector, 60% keyword (catches exact match)
)
```

2. Two-Stage Reranking

Fast retrieval (20 candidates) → Precise reranking (top 5):

- 20-30% precision improvement
- 3x latency (worth it for quality)

3. Systematic Failure Diagnosis

Automatic failure mode detection:

- No results retrieved → Check indexing
- Low relevance → Try query transformation
- Missing info → Increase top-k or improve chunking
- Hallucination → Strengthen grounding

4. Production Monitoring

Real-time tracking of:

- Retrieval quality (avg relevance score)
- Latency (P50, P95, P99)
- Failure modes (taxonomy)
- Success rate

Failure Mode Analysis

Analyzed 100 failing queries, identified root causes:

Failure Mode	Frequency	Fix	Improvement
Low relevance	45%	Hybrid search + reranking	82% resolved
Missing info	30%	Larger chunks, increase top-k	73% resolved
Hallucination	15%	Grounded prompts, citations	93% resolved
No results	10%	Query transformation (HyDE)	85% resolved

Evaluation

Test Suite

- 100 queries with ground truth
- 50 edge cases (ambiguous, multi-hop)

- 20 adversarial (designed to fail)

Metrics

- **Retrieval:** MRR, Precision@k, Recall@k, NDCG@k
- **Generation:** Faithfulness, Relevance, Correctness
- **E2E:** Answer quality, latency, cost

Usage

```
from production_rag import RAGPipeline

# Initialize
rag = RAGPipeline(
    retriever_type='hybrid',
    reranker=True,
    top_k=5
)

# Query
result = rag.query("How do I reset my password?")
print(result['answer'])
print(f"Confidence: {result['confidence']:.2f}")
print(f"Sources: {[d['metadata']['source'] for d in result['retrieved_docs']]}")
```

Design Decisions

Why Hybrid Search?

Vector-only misses exact matches (product codes, names).
BM25 catches these but misses semantic queries.
Hybrid gets best of both.

Tradeoff: 2x indexing cost, 20% latency increase.

Why Cross-Encoder Reranking?

Bi-encoder (vector search) is fast but imprecise.
Cross-encoder jointly encodes query+document → better relevance.

Tradeoff: 3x slower, but quality gain (25% precision) worth it.

Why Two-Stage Retrieval?

Can't rerank entire corpus (too slow).

Retrieve many candidates fast, rerank top candidates precisely.

Result: Best of both worlds - recall AND precision.

Production Lessons

1. 80% of failures are retrieval, not generation

- Fix: Invest in retrieval quality (hybrid, reranking)

1. Chunking strategy matters more than embedding model

- Fix: Experiment with chunk size, overlap

1. Monitor retrieval scores, not just answer quality

- Fix: Alert if avg relevance drops below 0.6

1. Hallucinations come from low-quality retrieval

- Fix: If retrieved docs are irrelevant, LLM makes things up

...

...

4. Blind Spots & Underrated Perspectives

What's Poorly Taught (Industry Reality Gaps)

Blind Spot 1: Most RAG Failures Are Retrieval Failures

Common teaching: "RAG = vector search + LLM generation"

Reality: 80% of quality issues stem from poor retrieval, not generation.

Evidence:

python

```
# Analyze 100 failing RAG queries
failures = analyze_failures(failing_queries)

# Root causes:
# Retrieval problems: 82%
#   - No relevant docs retrieved: 28%
#   - Low relevance scores: 35%
#   - Missing key information: 19%
# Generation problems: 18%
#   - Hallucination: 12%
#   - Poor synthesis: 6%
```

Why this matters:

- Teams spend 80% of effort on prompts (generation)
- Should spend 80% on retrieval quality
- Better retrieval = better answers (garbage in, garbage out)

How strong engineers think differently:

python

```
# Amateur approach: Focus on prompt engineering
BETTER_PROMPT = """
You are an expert assistant. Please provide a detailed,
accurate answer based on the context. Be thorough and precise...
"""

# Still fails if retrieved docs are irrelevant!

# Engineer approach: Fix retrieval first
1. Measure retrieval quality (MRR, Precision@k)
2. If MRR < 0.6 → retrieval problem, not prompt problem
3. Implement hybrid search, reranking, query transformation
4. Measure improvement
5. THEN optimize prompts
```

Production debugging workflow:

python

```
def debug_rag_failure(query, answer, expected):
    # Step 1: Is it a retrieval problem?
```

```

retrieved = retriever.search(query)

# Check if expected info is in retrieved docs
found_in_retrieved = any(
    expected_info_in_doc(doc, expected)
    for doc in retrieved
)

if not found_in_retrieved:
    print("RETRIEVAL PROBLEM: Expected info not retrieved")
    return "Fix retrieval (chunking, indexing, search strategy)"

# Step 2: Is expected info in context but answer is wrong?
if found_in_retrieved and answer != expected:
    print("GENERATION PROBLEM: Info available but answer wrong")
    return "Fix generation (prompt, model, grounding)"

# Step 3: Is latency/cost issue?
# etc.

```

Blind Spot 2: Chunk Size Is The Most Important Hyperparameter

Common teaching: "Use 512 tokens with 50 token overlap"

Reality: Optimal chunk size depends entirely on:

- Document structure
- Query types
- Context window size
- Precision vs recall tradeoff

Examples of when defaults fail:

python

```

# Case 1: Technical documentation
doc = """
API Endpoint: /api/users/create
Method: POST
Parameters:
  - name: string (required)
  - email: string (required)
  - role: string (optional, default: "user")
Response: {...}
"""

```

```

# With small chunks (200 tokens):
chunk1 = "API Endpoint: /api/users/create\nMethod: POST\n"
chunk2 = "Parameters:\n  - name: string (required)\n  - email..."
chunk3 = "Response: {...}"

# Problem: Parameters split from endpoint name!
# Query: "What parameters does user create endpoint take?"
# → Retrieves chunk2, but missing endpoint name context

# Fix: Larger chunks (500 tokens) that keep API definitions together

# Case 2: Narrative documents (novels, articles)
# Small chunks: Context lost, character/topic references unclear
# Large chunks: Better coherence, but more noise

# Case 3: FAQ documents
# Small chunks: Each Q&A pair is self-contained (good!)
# Large chunks: Multiple Q&As together (confusing)

```

Systematic chunk size tuning:

python

```

def evaluate_chunk_sizes(documents, queries, ground_truth):
    results = {}

    for chunk_size in [128, 256, 512, 1024, 2048]:
        for overlap in [0, 50, 100]:
            # Rechunk documents
            chunks = rechunk(documents, size=chunk_size, overlap=overlap)

            # Rebuild index
            retriever = VectorRetriever()
            retriever.add_documents(chunks)

            # Evaluate
            mrr = evaluate_retrieval(retriever, queries, ground_truth)

            results[(chunk_size, overlap)] = mrr

    best = max(results.items(), key=lambda x: x[1])
    print(f"Best: chunk_size={best[0][0]}, overlap={best[0][1]}, MRR={best[1]:.3f}")

# Example output:

```



```
# Default (512, 50): MRR=0.58
# Optimal (256, 0): MRR=0.72 (24% improvement!)
# Takeaway: Smaller chunks, no overlap works better for this corpus
```

How chunk size affects tradeoffs:

Small chunks (100-200 tokens):

- ✓ High precision (less noise in each chunk)
- ✓ Fits more results in context window
- ✗ Low recall (info split across chunks)
- ✗ Missing context (pronouns, references unclear)

Large chunks (1000-2000 tokens):

- ✓ High recall (captures full context)
- ✓ Better coherence (pronouns resolved)
- ✗ Low precision (more irrelevant info)
- ✗ Fewer chunks fit in context window
- ✗ Higher cost (more tokens)

Adaptive chunking (varies by document structure):

- ✓ Best of both worlds
- ✓ Respects semantic boundaries
- ✗ More complex to implement

Blind Spot 3: Reranking Is Not Optional (It's 2x Quality Improvement)

Common teaching: Not mentioned in most RAG tutorials.

Reality: Reranking is the single highest-ROI improvement for production RAG.

Why initial retrieval is imprecise:

python

```
# Vector search uses bi-encoder
query_emb = embed(query)      # Encode separately
doc_emb = embed(document)     # Encode separately
similarity = dot(query_emb, doc_emb) # Compare after

# Problem: Query and document encoded independently
# Can't capture query-document interactions

# Example where this fails:
query = "Python memory leak debugging"
doc1 = "Python memory profiling tools include memory_profiler and tracemalloc"
```

```
doc2 = "JavaScript garbage collection prevents memory leaks automatically."

# Vector search might rank doc2 higher (more overlap: "memory", "leak")
# But doc1 is actually more relevant (Python-specific tools)
```

Reranking with cross-encoder captures interactions:

python

```
# Cross-encoder sees query + document together
score = cross_encoder.predict([[query, doc]])

# Can capture:
# - Query-document relevance
# - Keyword importance
# - Semantic coherence
# - Answer-ability (does doc actually answer query?)
```

Measured impact:

python

```
# Benchmark on MS MARCO dataset
retrieval_only_MRR = 0.33      # Bi-encoder (BERT)
with_reranking_MRR = 0.42     # + Cross-encoder

# Improvement: 27% (huge!)

# On custom dataset (customer support)
baseline_precision@5 = 0.64
with_reranking_precision@5 = 0.87

# Improvement: 36%
```

When NOT to rerank:

- Latency-critical applications (<100ms requirement)
- Very large top-k (reranking 100+ docs is slow)
- Budget-constrained (cross-encoder inference costs)

Smart reranking strategy:

python

```

# Don't rerank on every query
def should_rerank(query, initial_results):
    # Rerank if:
    # 1. Low confidence in initial results
    if initial_results[0].score < 0.7:
        return True

    # 2. High-value query (business-critical)
    if is_high_value_query(query):
        return True

    # 3. Results are very similar (need fine-grained ranking)
    score_variance = np.std([r.score for r in initial_results[:5]])
    if score_variance < 0.05:
        return True

    return False

# Rerank selectively = 30% of queries
# Saves 70% of reranking cost while keeping quality high

```

Blind Spot 4: RAG Needs Different Evaluation Than Traditional QA

Common teaching: Use F1, exact match, BLEU.

Reality: RAG has unique failure modes requiring specific metrics.

Traditional QA metrics miss RAG issues:

python

```

# Example:
query = "What is the capital of France?"
ground_truth = "Paris"
rag_answer = "The capital of France is London. This is mentioned in the provided document"

# Traditional metrics:
f1_score = 0.0 # No word overlap
exact_match = False

# Problem: Metrics say answer is wrong, but don't tell us WHY
# Is it retrieval failure? Hallucination? Misunderstanding?

```

RAG-specific metrics:

python

```
class RAGMetrics:
    def evaluate(self, query, answer, retrieved_docs, ground_truth):
        return {
            # Retrieval quality
            'retrieval_mrr': self.calc_mrr(retrieved_docs),
            'retrieval_precision': self.calc_precision(retrieved_docs),

            # Answer quality
            'faithfulness': self.is_groundeds(answer, retrieved_docs), # NEW
            'answer_relevance': self.addresses_query(answer, query), # NEW
            'context_relevance': self.docs_relevant_to_query(retrieved_docs, query), #

            # Traditional
            'correctness': self.compare_to_truth(answer, ground_truth)
        }

    def is_groundeds(self, answer, docs):
        """Is answer supported by retrieved docs?"""
        # Hallucination detection
        claims = extract_claims(answer)
        doc_text = " ".join([d.text for d in docs])

        grounded_claims = [
            c for c in claims
            if claim_in_text(c, doc_text)
        ]

        return len(grounded_claims) / len(claims) if claims else 1.0

    def context_relevance(self, docs, query):
        """Are retrieved docs relevant to query?"""
        # Measures retrieval precision
        relevant_count = sum(
            1 for doc in docs
            if doc.score > 0.6 # Threshold
        )
        return relevant_count / len(docs) if docs else 0
```

RAG Eval Framework (RAGAS):

python

```

from ragas import evaluate
from ragas.metrics import (
    faithfulness,      # Answer grounded in docs?
    answer_relevancy,  # Answer addresses query?
    context_relevancy, # Docs relevant to query?
    context_recall     # Ground truth info in retrieved docs?
)

results = evaluate(
    dataset=eval_dataset,
    metrics=[faithfulness, answer_relevancy, context_relevancy, context_recall]
)

print(results)
# {
#   'faithfulness': 0.92,      # 92% of answer grounded
#   'answer_relevancy': 0.87,  # 87% relevant to query
#   'context_relevancy': 0.81, # 81% of docs relevant
#   'context_recall': 0.89     # 89% of ground truth retrieved
# }

# Can diagnose issues:
# - Low context_recall → retrieval problem (missing info)
# - Low faithfulness → hallucination problem
# - Low answer_relevancy → prompt/generation problem

```

Why this matters: Without RAG-specific metrics, you're flying blind.

Blind Spot 5: Query Transformation Is Underrated

Common teaching: "Embed the user query and search."

Reality: User queries are often poor search queries. Transform them first.

Why queries fail:

python

```

# User queries are:
# 1. Verbose
"I'm having trouble with my Python code where I'm trying to read a CSV file but it keeps

# 2. Ambiguous
"How do I install it?" # Install what?

```

```
# 3. Questions (docs are answers)
"What is the syntax for list comprehension?"
# But docs say: "List comprehension syntax: [x for x in range(10)]"
# Query-document mismatch!

# 4. Missing context
"How do I reset it?" # Reset what? Password? Device? Settings?
```

Query transformation strategies:

python

```
# Strategy 1: Query condensation
verbose_query = "I'm having trouble with my Python code where..."
condensed = llm.complete(f"Condense to key terms: {verbose_query}")
# "Python CSV encoding error"

# Strategy 2: HyDE (Hypothetical Document Embeddings)
query = "How do I debug memory leaks in Python?"
hypothetical_answer = llm.complete(f"Answer this question: {query}")
# "To debug memory leaks in Python, use memory_profiler or tracemalloc..."

# Search with hypothetical answer (closer to doc style)
results = vector_db.search(hypothetical_answer)

# Strategy 3: Query expansion
query = "password reset"
expanded = llm.complete(f"Related search terms for: {query}")
# ["password reset", "forgot password", "change password", "reset credentials"]

# Search multiple variants, merge results

# Strategy 4: Multi-query
query = "Compare Python and JavaScript performance"
sub_queries = llm.complete(f"Break into sub-queries: {query}")
# ["Python performance benchmarks", "JavaScript performance benchmarks",
#   "Python vs JavaScript speed comparison"]

# Search each, combine results
```

Impact:

python

```
# Benchmark: 100 verbose/ambiguous queries
```

```
baseline_MRR = 0.54 # Search raw query
```

```
with_condensation_MRR = 0.63 # +17%
```

```
with_HyDE_MRR = 0.68 # +26%
```

```
with_multi_query_MRR = 0.71 # +31%
```

```
# Best: All three combined (pipeline)
```

```
# 1. Condense
```

```
# 2. Generate hypothetical answer (HyDE)
```

```
# 3. Create sub-queries
```

```
# 4. Search all variants
```

```
# 5. Deduplicate and rerank
```

```
final_MRR = 0.76 # +41% vs baseline
```

How This Perspective Prevents Failures

Fragile RAG (Amateur):

- Vector search only (no reranking)
- Fixed chunk size (512 tokens, always)
- No query transformation
- No retrieval metrics (can't measure quality)
- Prompt engineering as first resort
- **Result:** 60-70% answer quality, can't debug failures

Robust RAG (Engineer):

- Hybrid search + cross-encoder reranking
- Adaptive chunking (based on document structure)
- Query transformation pipeline
- Comprehensive metrics (retrieval + generation + E2E)
- Systematic failure diagnosis
- **Result:** 90%+ answer quality, debuggable, maintainable